Module 1: Angular Fundamentals & Architecture

A comprehensive introduction to the Angular framework — covering core architecture, ecosystem, project structure, and building blocks for enterprise-grade web development.

90 MINUTES · 12 SLIDES



Learning Objectives

By the end of this module, you will be able to:

01

Understand the Angular Ecosystem

Recognize the full suite of tools Angular provides out of the box.

02

Explain Angular Architecture

Describe how Angular layers work together — from components to backend services.

03

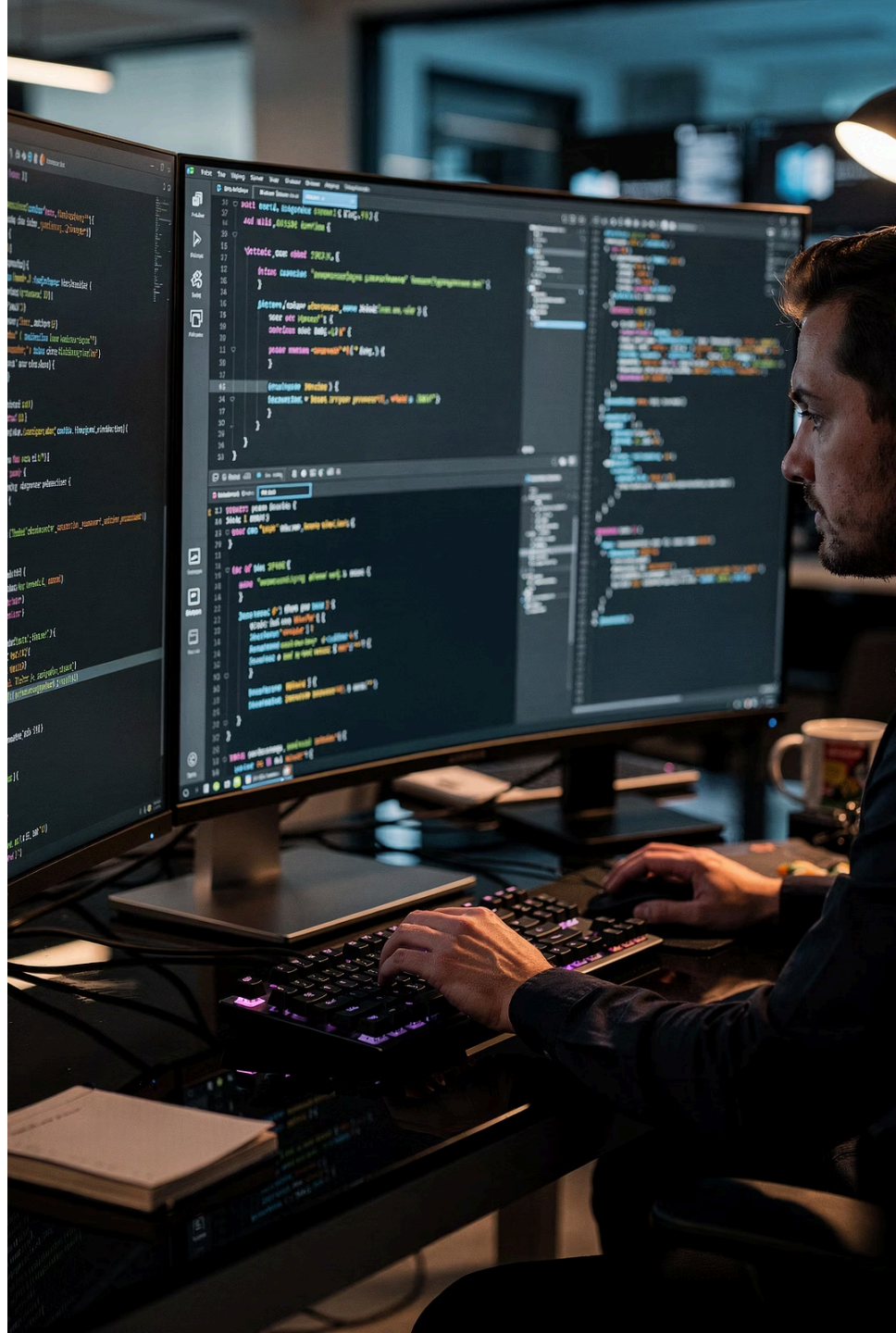
Explore Project Structure

Navigate an Angular workspace and understand each key file and folder.

04

Identify Core Building Blocks

Distinguish between components, services, directives, templates, and pipes.





FRAMEWORK OVERVIEW

What is Angular?

Angular is a **TypeScript-based frontend framework** developed and maintained by **Google** for building modern, scalable, and maintainable web applications — not just a UI library, but a *complete application framework*.

Typical apps: Enterprise Portals, Banking Systems, CRM, ERP, Healthcare, E-Commerce

Component-Based

Cross Platform

Enterprise Ready

TypeScript First

Why Angular for Enterprise?

Modern enterprise applications demand more than UI rendering. Angular is purpose-built to meet those demands.

What Enterprise Apps Require

Scalability & Maintainability

Structure that holds up as teams and codebases grow.

Security & Performance

Built-in guards, interceptors, and optimized rendering.

Team Collaboration

Consistent patterns that align distributed engineering teams.

How Angular Delivers

Component Architecture

Modular, reusable UI units

Dependency Injection

Decoupled, testable services

CLI Tooling

Faster, consistent development

Built-in Routing & Forms

No third-party setup required

The Angular Ecosystem

Unlike most frontend libraries, Angular ships with nearly everything an enterprise team needs — no assembly required.



Angular CLI

Scaffold, build, test, and deploy with a single tool.



Router & Forms

Built-in routing and reactive/template-driven forms.



HttpClient & RxJS

Declarative async data handling and HTTP communication.



Signals

Modern reactive state management built into Angular core.



Angular Material

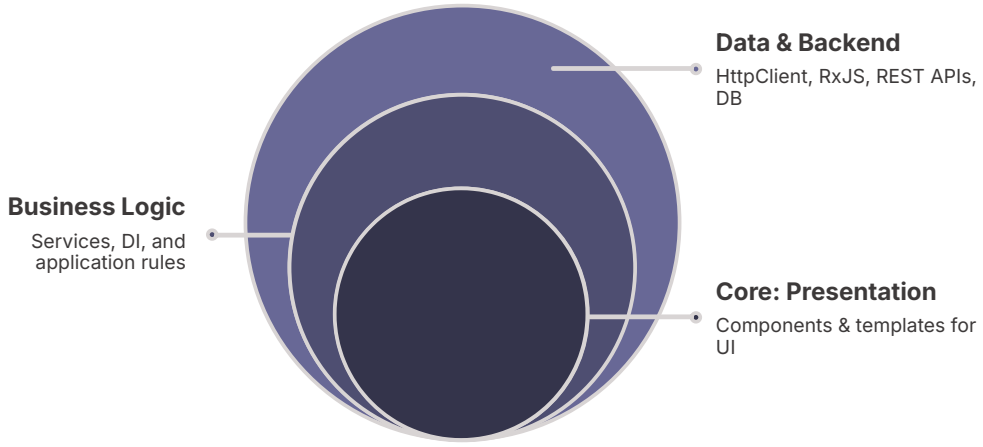
Pre-built, accessible UI components aligned to Material Design.



Dependency Injection

Hierarchical DI system for scalable service architecture.

Angular Architecture



Why Layered Architecture?

Loose Coupling

Each layer operates independently, reducing interdependencies.

Better Testability

Isolate and unit test each layer without side effects.

Easy Maintenance

Changes in one layer don't cascade unpredictably.

Reusable Logic

Services shared across components without duplication.

Angular vs Vue

Both are excellent frameworks — but Angular is purpose-built for large-scale enterprise applications.

Feature	Angular	Vue
Type	Full Framework	Progressive Framework
Language	TypeScript	JavaScript
Learning Curve	Moderate	Easy
Built-in Features	Extensive	Minimal
Enterprise Support	Excellent	Good
Scalability	High	Medium

Choose Angular when: Large teams, long-term maintenance, complex business logic, or strict enterprise governance requirements are in play.

What's New in Angular 21?

Angular 21 continues its rapid evolution — reducing boilerplate, boosting performance, and improving the overall developer experience.

Standalone Components

No NgModules required — simpler, self-contained component design.

Signals

Fine-grained reactivity built directly into Angular core.

Modern Control Flow

New `@if`, `@for`, and `@switch` block syntax replaces structural directives.

Deferred Loading

Lazy-load UI sections declaratively with `@defer`.

Improved SSR

Better server-side rendering with hydration support.

AI-Friendly Dev

Cleaner syntax and patterns that work better with AI code assistants.

Angular Project Structure

```
src/
├── app/          ← Application source code
│   ├── app.component.ts
│   ├── app.routes.ts
│   └── app.config.ts
├── assets/       ← Images, fonts, icons
├── styles/       ← Global stylesheets
├── index.html    ← App shell
└── main.ts       ← Entry point

angular.json      ← CLI configuration
package.json      ← Dependencies & scripts
tsconfig.json     ← TypeScript settings
```

Key Files Explained

→ main.ts

Bootstraps the application using `bootstrapApplication()` with the root component.

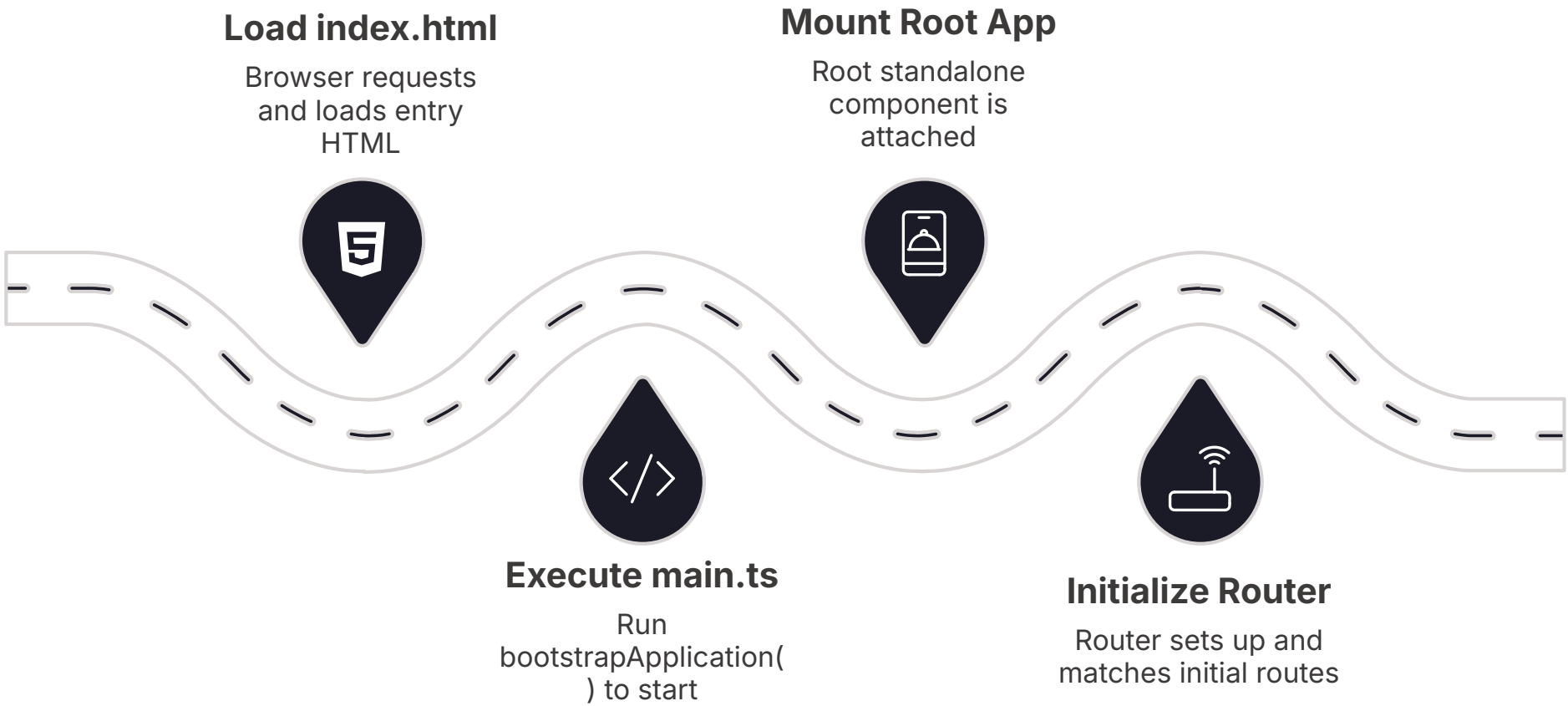
→ app.config.ts

Registers global providers — router, HTTP client, and interceptors.

→ angular.json

Controls build targets, environments, and asset configuration for the CLI.

Angular Application Bootstrap Process



Modern Angular apps use `bootstrapApplication()` with standalone components — replacing the older `AppModule`-based approach. This simplifies startup, reduces boilerplate, and makes the initialization sequence easier to trace and test.

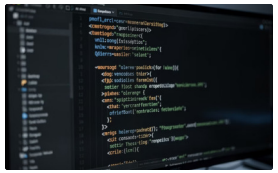
Angular Building Blocks

Every Angular application is composed of five fundamental building blocks. Together, they create modular, reusable, and maintainable applications.



Component

Encapsulates a piece of UI — such as a login form, dashboard panel, or data table.



Template

Defines the view using HTML combined with Angular binding syntax and directives.



Directive

Modifies the appearance or behavior of DOM elements — structurally or as attributes.



Service

Contains reusable business logic and API communication, injected where needed.



Pipe

Transforms data before display — dates, currency, custom formatting, and more.

Module 1 Summary

What We Covered

- Angular overview and key characteristics
- Why Angular suits enterprise development
- The Angular ecosystem and built-in tools
- Layered application architecture
- Angular vs Vue comparison
- New features in Angular 21
- Project structure and bootstrap process
- Core building blocks

Next Module

Components & Templates

Build reusable user interfaces using modern standalone components. Learn component composition, data binding, and template syntax.